Projet BI – Rapport

Data warehouse des ventes classicmodels et dellstore

Groupe 5

Neama HMAMI, Antoine DUSART, Lounes CHIKH

Business Intelligence – Dr S. Cremer

9 novembre 2026

Table des matières

1 Contexte et architecture **2**

2 Les 5 étapes d'analyse **2**

3 Schéma de la BD OLAP (étoile) **3**

4 Schéma de la BD de staging (MySQL) **3**

5 Flux réalisé dans NiFi **4**

6 Configuration des processeurs **6**

7 Règles de transformation **7**

8 Visualisation Power BI **8**

9 Bonus : Change Data Capture, contrôles et limites **10**

A Fichiers fournis **14**

1 Contexte et architecture

Les ventes de deux bases OLTP (classicmodels et dellstore) sont réunies chaque nuit dans une BD OLAP en étoile, analysée dans Power BI Desktop.

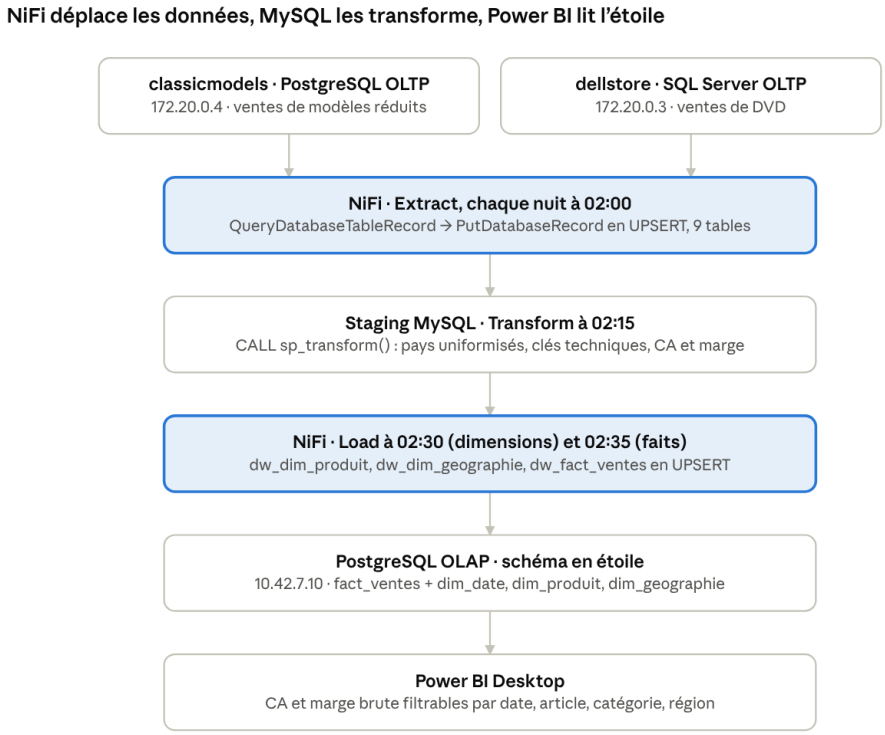


FIGURE 1 – Architecture du projet : 2 sources, 1 ETL (NiFi), 1 staging (MySQL), 1 OLAP (PostgreSQL)

Les bases OLTP ne sont jamais modifiées : NiFi les lit, et toute la transformation se fait dans le staging MySQL (slide 43), avant le chargement de l'OLAP.

2 Les 5 étapes d'analyse

Le modèle retenu est une étoile au grain de la ligne de commande, avec trois dimensions : date, produit, géographie (démarche de la slide 70).

1. **Identifier le business process** : les ventes des deux sociétés (modèles réduits pour classicmodels, DVD pour dellstore). But : suivre le chiffre d'affaires (CA) et la marge brute, filtrables par date, article, catégorie d'article et région géographique.
2. **Définir le grain** : une ligne de commande (un article dans une commande), au jour près. C'est le niveau le plus fin des deux sources ; tout le reste s'obtient par agrégation (roll-up), jamais l'inverse.
3. **Identifier les dimensions** : Date (quand), Produit (quoi : article et catégorie), Géographie (où : zone, pays, état, ville du client). Date est une dimension statique générée en SQL ;

Produit et Géographie sont des SCD type 1. Source, numéro de commande, numéro de ligne et statut restent dans la table de faits comme dimensions dégénérées (slide 80).

4. **Définir les faits** : quantité, CA (quantité × prix unitaire), coût d'achat (quantité × coût unitaire), marge brute (CA − coût d'achat). Les quatre mesures sont additives ; le taux de marge, non additif, est calculé dans Power BI.
5. **Créer le schéma** : étoile, avec une table de faits et trois dimensions dénormalisées (la catégorie est dans `dim_produit`, la zone et le pays dans `dim_geographie`). Les requêtes restent simples, avec une seule jointure par dimension (slide 74).

3 Schéma de la BD OLAP (étoile)

La BD OLAP PostgreSQL contient quatre tables : `fact_ventes` au centre, reliée par clé étrangère à `dim_date`, `dim_produit` et `dim_geographie`.

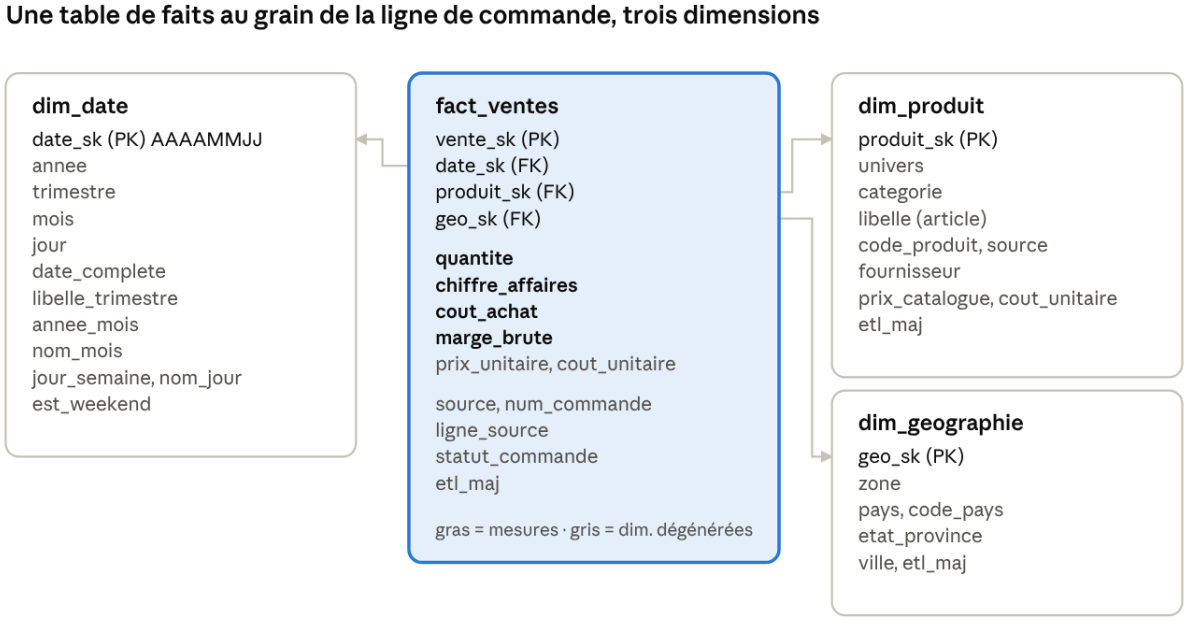


FIGURE 2 – Schéma en étoile de la BD OLAP : 1 table de faits, 3 dimensions

Les colonnes de chaque dimension suivent leur hiérarchie (slide 69) : année > trimestre > mois > jour, univers > catégorie > article, zone > pays > état > ville. La clé de `dim_date` est au format AAAAMMJJ et la table est générée par SQL de 2000 à 2030 ; les autres clés techniques viennent du staging. Script : `01_olap_postgres.sql`.

4 Schéma de la BD de staging (MySQL)

La transformation se fait dans MySQL (zone de transit, slide 43), pas dans NiFi : NiFi ne fait que déplacer les données, et la procédure stockée `sp_transform` construit les tables finales.

Groupe	Tables	Remplies par	Rôle
`stg_cm_*`	`stg_cm_products`, `stg_cm_customers`, `stg_cm_orders`, `stg_cm_orderdetails`	NiFi (UPSERT)	Copie de classicmodels, colonnes utiles uniquement
`stg_ds_*`	`stg_ds_categories`, `stg_ds_products`, `stg_ds_customers`, `stg_ds_orders`, `stg_ds_orderlines`	NiFi (UPSERT)	Copie de dellstore, colonnes utiles uniquement
`ref_*`	`ref_pays`, `ref_taux_cout_ds`, `etl_parametre`	Script SQL 02	Correspondance pays → nom unique + zone ; taux de coût dellstore
`v_clients`	Vue	–	Clients des 2 sources avec une géographie uniformisée
`dw_*`	`dw_dim_produit`, `dw_dim_geographie`, `dw_fact_ventes`	`sp_transform`	Contenu final de l'OLAP, avec les clés techniques (AUTO_INCREMENT)
`etl_*`	`etl_controle`, `etl_journal`	`sp_transform`	Watermark du CDC, journal de chaque exécution

TABLE 1 – Tables de la BD de staging

Chaque table `stg_` et `dw_` porte une colonne `etl_maj DATETIME(3) ... ON UPDATE CURRENT_TIMESTAMP(3)`. MySQL ne la met à jour que si une valeur de la ligne change réellement : c'est elle qui repère les lignes modifiées (CDC). Scripts : `02_staging_mysql.sql` (tables, références, vue) et `03_transform_mysql.sql` (procédure).

5 Flux réalisé dans NiFi

Le process group « Projet BI » contient 4 sous-groupes, soit 26 processeurs, déclenchés chaque nuit dans l'ordre Extract → Transform → Load.

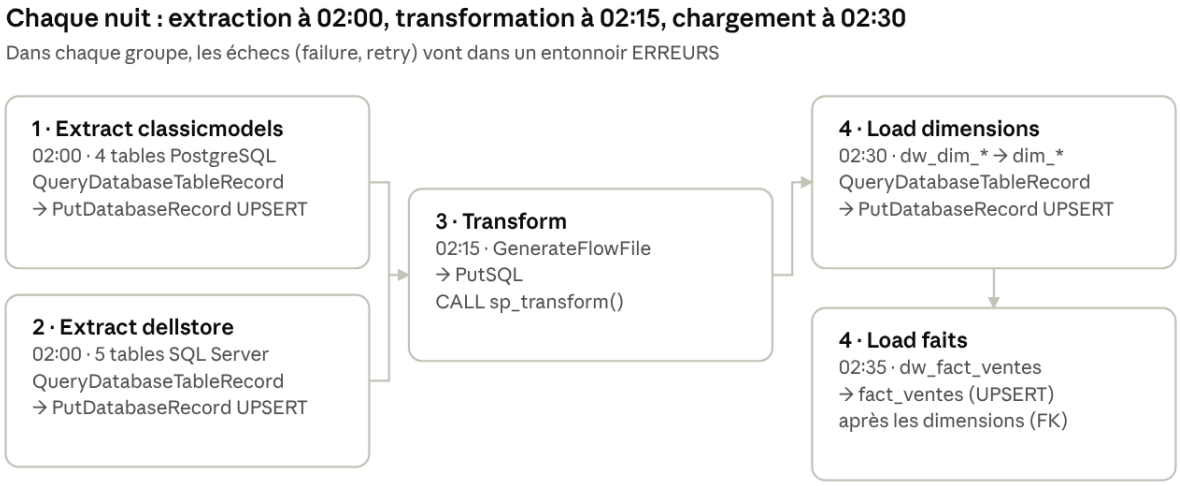


FIGURE 3 – Flux NiFi : 4 process groups planifiés en CRON

Le décalage horaire garantit l'ordre : les dimensions arrivent dans l'OLAP avant les faits qui les référencent. Pour une démonstration, on lance chaque groupe à la main (clic droit > Run Once) dans le même ordre. Le flux complet est fourni dans `flux_projet_bi.json`, à importer dans NiFi. Les captures NiFi ci-dessous viennent du NiFi de l'école (version 1.23.1, groupe 5). Elles ont été prises juste après une exécution complète du flux sur les vraies bases : 106 966 lignes extraites des deux OLTP et 64 106 ventes chargées dans l'OLAP en 1 min 25 s, sans aucune erreur.

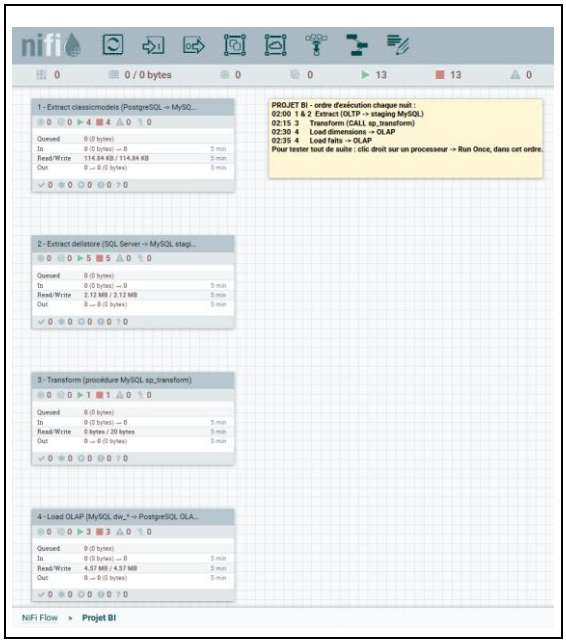


FIGURE 4 – Vue d'ensemble du process group Projet BI

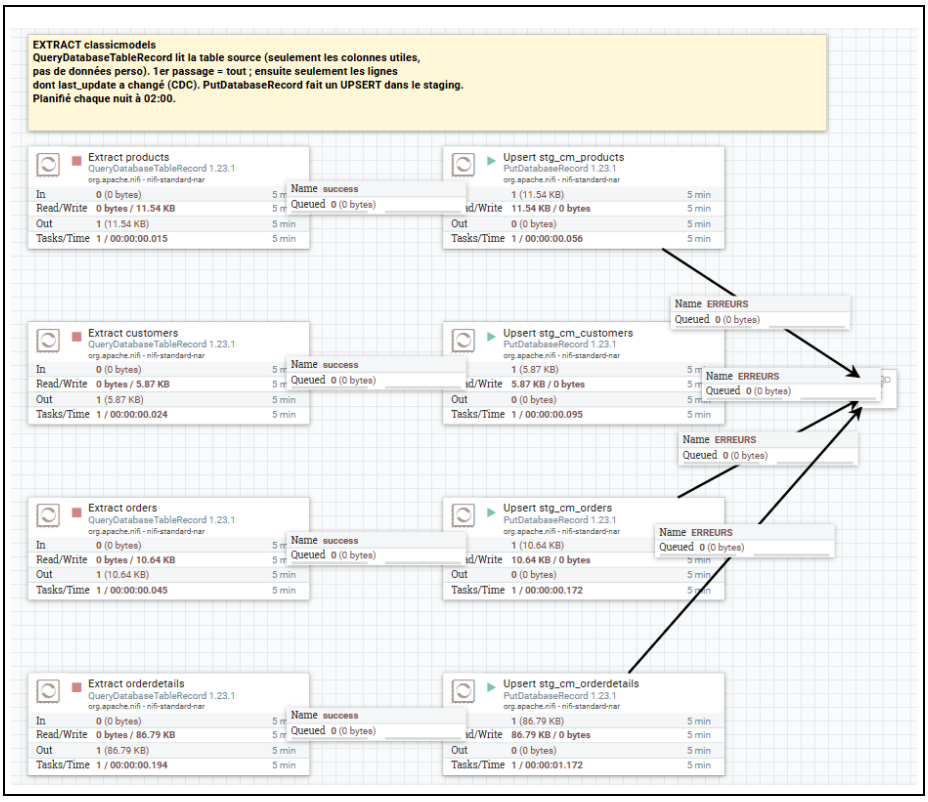


FIGURE 5 – Intérieur du groupe 1 - Extract classicmodels

6 Configuration des processeurs

Les connexions sont des controller services du groupe « Projet BI ». Leurs URL, logins et mots de passe viennent du contexte de paramètres « Projet BI - connexions ».

Controller service	**URL / classe du driver / emplacement du driver**

DBCP classicmodels	`jdbc:postgresql://172.20.0.4:5432/classicmodels`
	`org.postgresql.Driver`
	`/opt/nifi/drivers/postgresql-42.6.0.jar`

DBCP dellstore	`jdbc:sqlserver://172.20.0.3:1433;encrypt=false;databaseName=dellstore;integratedSecurity=false;`
	`com.microsoft.sqlserver.jdbc.SQLServerDriver`
	`/opt/nifi/drivers/mssql-jdbc-12.4.0.jre8.jar`

DBCP staging	`jdbc:mysql://172.20.0.65:3306/user_ktm`
	`com.mysql.cj.jdbc.Driver`
	`/opt/nifi/drivers/mysql-connector-j-8.1.0.jar`

DBCP OLAP	`jdbc:postgresql://10.42.7.10:5432/user_ktm`
	`org.postgresql.Driver`
	`/opt/nifi/drivers/postgresql-42.6.0.jar`

AvroRecordSetWriter, AvroReader	Valeurs par défaut (schéma Avro embarqué)

TABLE 2 – Controller services

Processeur (nombre)	**Propriétés modifiées**	**Planification**
QueryDatabase-TableRecord, extraction (9)	Database Type = PostgreSQL ou MS SQL 2012+ ; Table Name ; Columns to Return (classicmodels) ou Custom Query (dellstore) ; Maximum-value Columns = `last_update` ; Record Writer = AvroRecordSetWriter ; Use Avro Logical Types = true	CRON `0 0 2 * * ?`
PutDatabaseRecord, vers le staging (9)	Record Reader = AvroReader ; Database Type = MySQL ; Statement Type = UPSERT ; DBCP staging ; Table Name = `stg_...` ; Unmatched Column Behavior = Ignore Unmatched Columns	En continu ; success auto-terminé ; failure et retry vers l'entonnoir ERREURS
GenerateFlowFile (1)	Custom Text = `CALL sp_transform();` (simple déclencheur)	CRON `0 15 2 * * ?`
PutSQL (1)	JDBC Connection Pool = DBCP staging ; SQL Statement = `CALL sp_transform()`	En continu
QueryDatabase-TableRecord, lecture `dw_` (3)	Database Type = MySQL ; Table Name = `dw_...` ; Maximum-value Columns = `etl_maj`	Dimensions `0 30 2 * * ?`, faits `0 35 2 * * ?`
PutDatabaseRecord, vers l'OLAP (3)	Database Type = PostgreSQL ; Statement Type = UPSERT ; DBCP OLAP ; Table Name = `dim_produit`, `dim_geographie`, `fact_ventes` ; Update Keys = `produit_sk`, `geo_sk`, `vente_sk`	En continu

TABLE 3 – Processeurs NiFi

Exemple de Custom Query dellstore (la date de modification vient de la table `hist_`) :

```
SELECT ol.orderid, ol.orderlineid, ol.prod_id, ol.quantity, ol.orderdate,
       COALESCE(h.last_update, CAST('1900-01-01' AS DATETIME2(3))) AS last_update
FROM orderlines ol
LEFT JOIN hist_orderlines h ON h.orderid = ol.orderid AND h.orderlineid = ol.orderlineid
```

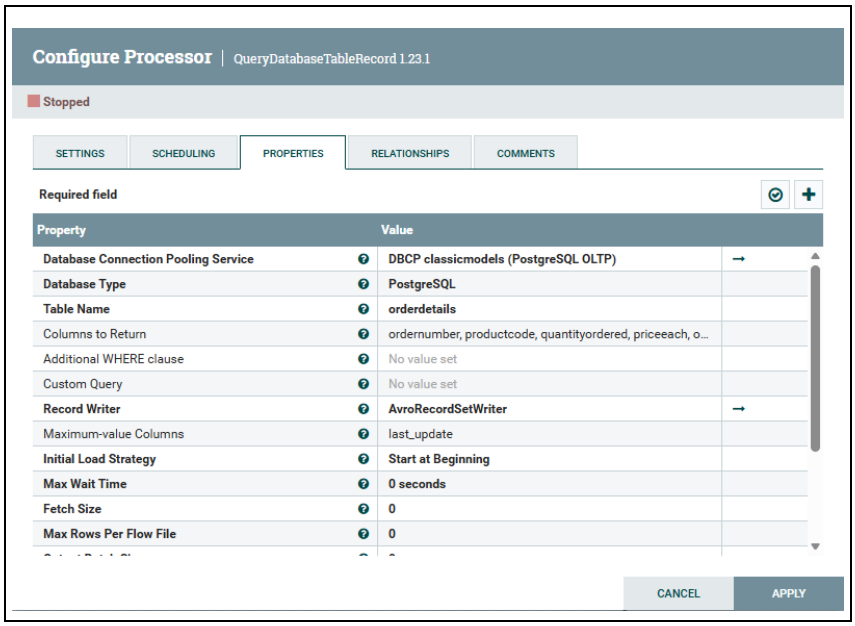


FIGURE 6 – Fenêtre Configure d'un QueryDatabaseTableRecord (Extract orderdetails)

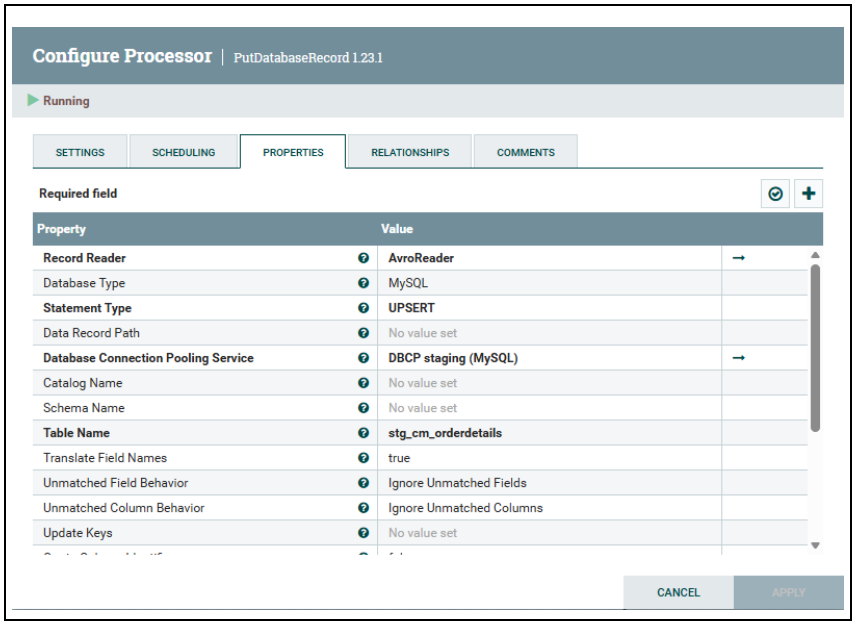


FIGURE 7 – Fenêtre Configure d'un PutDatabaseRecord (Upsert stg_cm_orderdetails)

7 Règles de transformation

La procédure `sp_transform` applique les règles suivantes dans une seule transaction (tout ou rien) :

— **Uniformisation des pays** (slides 46 et 48) : la table `ref_pays` ramène « USA » et « US » à United States, « UK » à United Kingdom, et « Norway » suivi d'espaces à Norway. Chaque pays reçoit une zone : Europe, Amérique du Nord, Amérique latine, Asie-Pacifique, Afrique ou Moyen-Orient. Les clients dellstore viennent de 246 pays : `ref_pays` contient donc la liste mondiale complète (262 libellés). Un pays inconnu recevrait la zone « Autre » ; après chargement, aucun client n'est dans ce cas.

— **Champs vides** : un état ou une ville vide devient « N/A », et un statut absent (dellstore) « Non renseigné ».

— **Clés** : chaque ligne de dimension reçoit une clé technique, et la clé métier (source + code) est unique. Le produit 1 de dellstore ne peut donc pas être confondu avec un produit classicmodels. Une vente est identifiée par (source, numéro de commande, ligne).

— **CA et coût classicmodels** : CA = `quantityordered` × `priceeach` (le prix réel de la ligne), coût = `quantityordered` × `buyprice`.

— **CA et coût dellstore** : `orderlines` n'a pas de prix, donc CA = `quantity` × `products.price`. dellstore n'a **aucun prix d'achat** : le coût est estimé à 65 % du prix de vente. Ce taux est réglable dans `etl_parametre`, ou par catégorie dans `ref_taux_cout_ds`.

— **Anonymisation** (slide 29) : aucune donnée personnelle n'est extraite (noms, adresses, téléphones, cartes de crédit, mots de passe). Seuls la ville, l'état et le pays du client sont conservés.

— **Commandes annulées** : elles restent dans les faits avec leur statut (« Cancelled ») et sont exclues des mesures Power BI.

— **Date** : la date de commande donne `date_sk` au format AAAAMMJJ (par exemple 20040315).

8 Visualisation Power BI

Power BI Desktop lit directement l'étoile de la BD OLAP (connecteur PostgreSQL, mode Import, serveur `10.1.20.51:27171` depuis le local de cours). Le rapport est fourni sous forme de projet Power BI (`powerbi/ProjetBI.pbip`) : le serveur et la base sont deux paramètres Power Query (`Serveur`, `BaseOLAP`), modifiables sans toucher au modèle.

Modèle. `fact_ventes` est reliée à `dim_date`, `dim_produit` et `dim_geographie` par `date_sk`, `produit_sk` et `geo_sk` (plusieurs-à-un, filtre à sens unique). `dim_date` est marquée comme table de dates sur `date_complete`, et `nom_mois` est trié par `mois`. Trois hiérarchies permettent le drill-

down : Année > Trimestre > Mois > Jour, Univers > Catégorie > Article, Zone > Pays > État > Ville.

Mesures DAX (les commandes annulées ne comptent pas) :

```
CA =
CALCULATE ( SUM ( fact_ventes[chiffre_affaires] ),
            fact_ventes[statut_commande] <> "Cancelled" )

Marge brute =
CALCULATE ( SUM ( fact_ventes[marge_brute] ),
            fact_ventes[statut_commande] <> "Cancelled" )

Taux de marge = DIVIDE ( [Marge brute], [CA] )

CA N-1 = CALCULATE ( [CA], SAMEPERIODLASTYEAR ( dim_date[date_complete] ) )
```

Visuel	Champs	Rôle
4 segments (slicers)	Période (entre deux dates), Région (Zone > Pays), Catégorie et Article (listes déroulantes)	Filtres demandés : date, article, catégorie, région
4 cartes	CA, Marge brute, Taux de marge, Quantité vendue	Indicateurs clés
Courbe	Mois (`annee_mois`) en axe ; CA et Marge brute	Évolution dans le temps
Barres groupées	Catégorie ; CA et Marge brute, triées par CA	Comparaison des catégories
Histogramme groupé	Zone ; CA et Marge brute	Répartition par région
Matrice	Lignes Zone > Pays, colonnes Année ; CA, Marge brute	Roll-up, drill-down et pivot (slides 57 à 61)

TABLE 4 – Page « Analyse des ventes »

Les captures ci-dessous ont été prises dans Power BI Desktop (version 2.159), connecté à la BD OLAP de l'école. Le total de la matrice (CA 16 006 097, marge 6 307 401) correspond au contrôle SQL de la BD OLAP, commandes annulées exclues. Filtrée sur Europe et Classic Cars, la page donne un CA de 2 522 892 et une marge de 1 018 843, là encore identiques au SQL. Le serveur PostgreSQL de l'école utilise un certificat auto-signé : il faut décocher « Chiffrer les connexions » dans les paramètres de la source de données de Power BI.

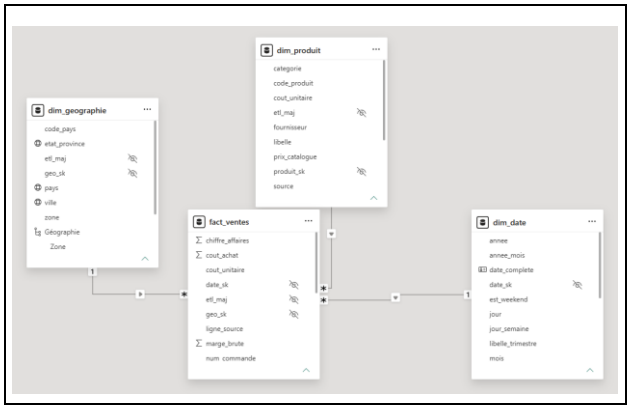
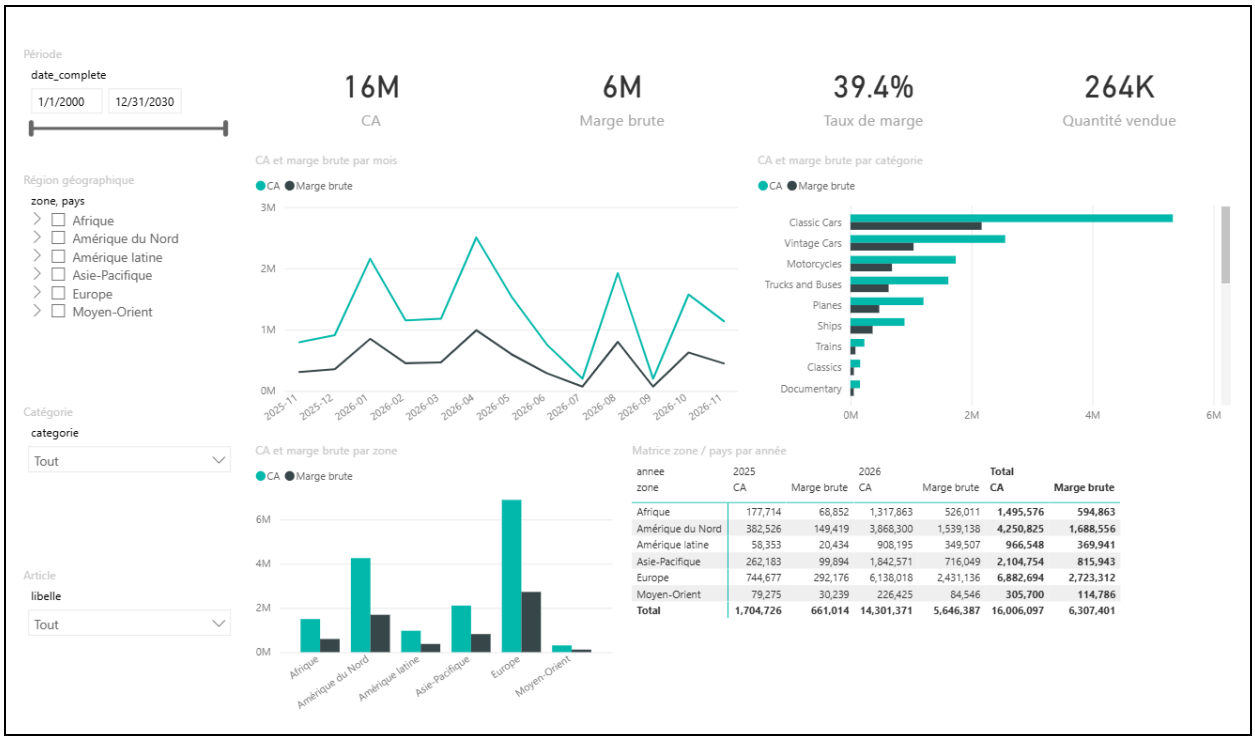
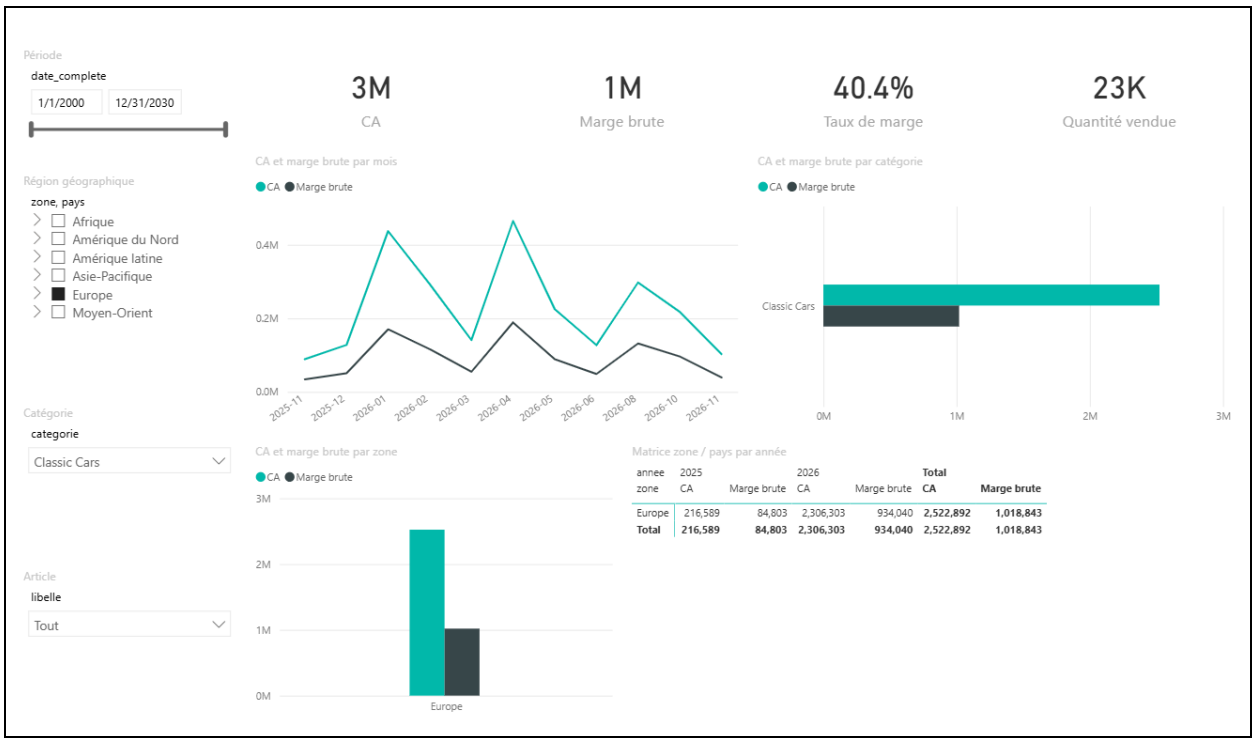


FIGURE 8 – Vue Modèle : l'étoile et ses 3 relations plusieurs-à-un



annee	2025		2026		Total	
zone	CA	Marge brute	CA	Marge brute	CA	Marge brute
Afrique	177,714	68,852	1,317,863	526,011	**1,495,576**	**594,863**
Amérique du Nord	382,526	149,419	3,868,300	1,539,138	**4,250,825**	**1,688,556**
Amérique latine	58,353	20,434	908,195	349,507	**966,548**	**369,941**
Asie-Pacifique	262,183	99,894	1,842,571	716,049	**2,104,754**	**815,943**
Europe	744,677	292,176	6,138,018	2,431,136	**6,882,694**	**2,723,312**
Moyen-Orient	79,275	30,239	226,425	84,546	**305,700**	**114,786**
Total	**1,704,726**	**661,014**	**14,301,371**	**5,646,387**	**16,006,097**	**6,307,401**

FIGURE 9 – Page « Analyse des ventes », sans filtre



annee	2025		2026		Total	
zone	CA	Marge brute	CA	Marge brute	CA	Marge brute
Europe	216,589	84,803	2,306,303	934,040	**2,522,892**	**1,018,843**
Total	**216,589**	**84,803**	**2,306,303**	**934,040**	**2,522,892**	**1,018,843**

FIGURE 10 – La même page filtrée sur Europe et Classic Cars

9 Bonus : Change Data Capture, contrôles et limites

Le pipeline ne déplace que ce qui a changé, à chacune des trois étapes (slides 88 et 89) :

1. **Extract** : QueryDatabaseTableRecord mémorise le plus grand `last_update` déjà lu. Le premier passage charge tout (initial load), les suivants uniquement les lignes plus récentes.

classicmodels a une colonne `last_update` dans ses tables ; dellstore garde ses dates de modification dans les tables `hist_*`.

2. **Transform** : `sp_transform` ne recalcule que les lignes dont `etl_maj` est postérieur au dernier passage (watermark dans `etl_controle`). Les lignes jamais intégrées sont aussi reprises, par sécurité. Les ventes déjà chargées ne sont jamais recalculées, ce qui respecte la non-volatilité (slide 23).
3. **Load** : seules les lignes `dw_` dont `etl_maj` a changé sont envoyées vers l'OLAP, en UPSERT.

Démonstration sur l'infrastructure de l'école. Les deux OLTP sont partagés entre les groupes et en lecture seule. C'est le professeur qui y ajoute des lots de modifications, le dernier le 09/11 à 11:09 : 8 commandes et 40 lignes par source, plus quelques clients et produits mis à jour. Ce lot a été rejoué sans toucher aux sources :

1. l'entrepôt du groupe (staging et OLAP) a été remis dans son état d'avant le lot, en retirant ses 16 commandes et leurs 80 ventes ;
2. le repère CDC de NiFi a été placé juste avant le lot, avec la propriété `initial.maxvalue.last_update` = `2026-11-09 11:00:00` ;
3. l'ETL a ensuite été lancé normalement.

Seul le lot a circulé. L'extraction `orderdetails` pèse 1,47 Ko, contre 86,79 Ko au chargement complet. La transformation a traité exactement 40 + 40 ventes en 2,3 s. Aucune dimension n'a été renvoyée, car les clients et produits du lot étaient déjà identiques. Les 80 ventes ont été chargées en un seul envoi, et l'OLAP est revenu exactement aux totaux des sources. Les passages sans changement ne recalculent rien, même après une ré-extraction complète : l'UPSERT d'une ligne identique ne modifie pas `etl_maj`.

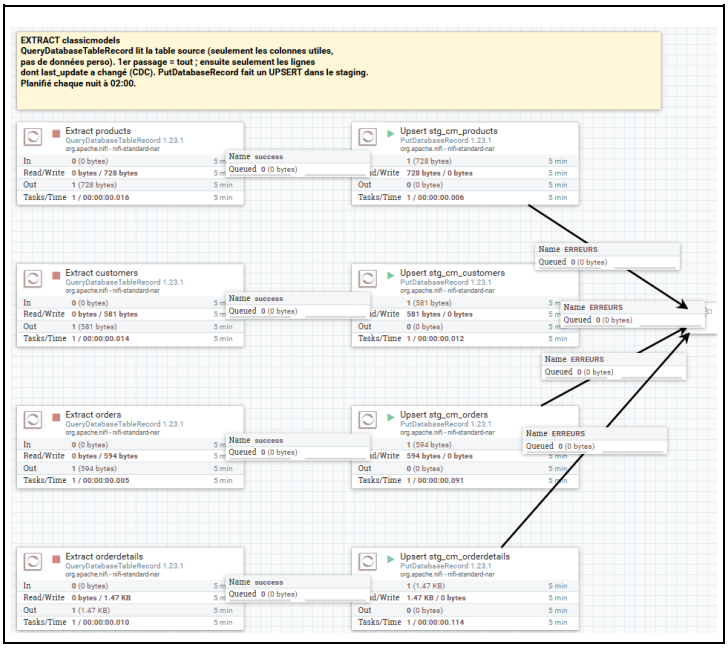


FIGURE 11 – Passage incrémental dans le groupe 1 : seul le lot du 09/11 est extrait

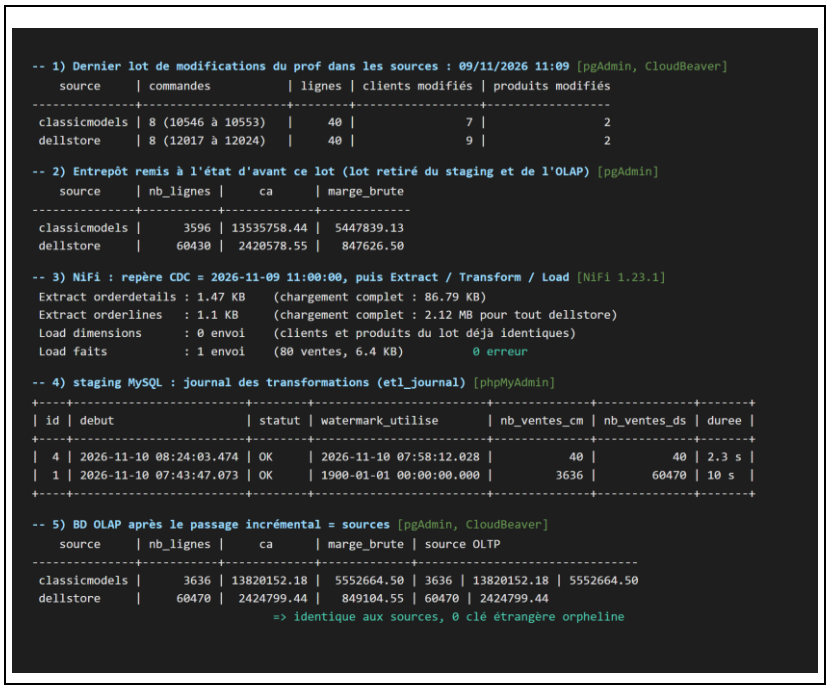


FIGURE 12 – Rejeu du lot du 09/11 sur l'infrastructure de l'école : seul le lot circule, OLAP = sources

Les cas que le professeur ne produit pas (quantité modifiée, commande annulée) ont été testés sur une copie locale des deux schémas (Docker : PostgreSQL, SQL Server, MySQL et NiFi 1.28) :

Exécution	**Changements dans les sources**	**Ventes envoyées vers l'OLAP**	**Totaux OLAP = sources**
1 (chargement initial)	–	701	Oui
2	Aucun	0	Oui
3	1 quantité modifiée, 1 commande annulée, 2 nouvelles commandes, 1 nouveau client	7	Oui

TABLE 5 – Test du CDC

Contrôles post-chargement (slide 49, script `04_controles.sql`) : CA et marge brute recalculés dans chaque source et comparés à l'OLAP, pays non reconnus, lignes de commande non transformées, et journal `etl_journal` de chaque exécution (OK ou ERREUR).

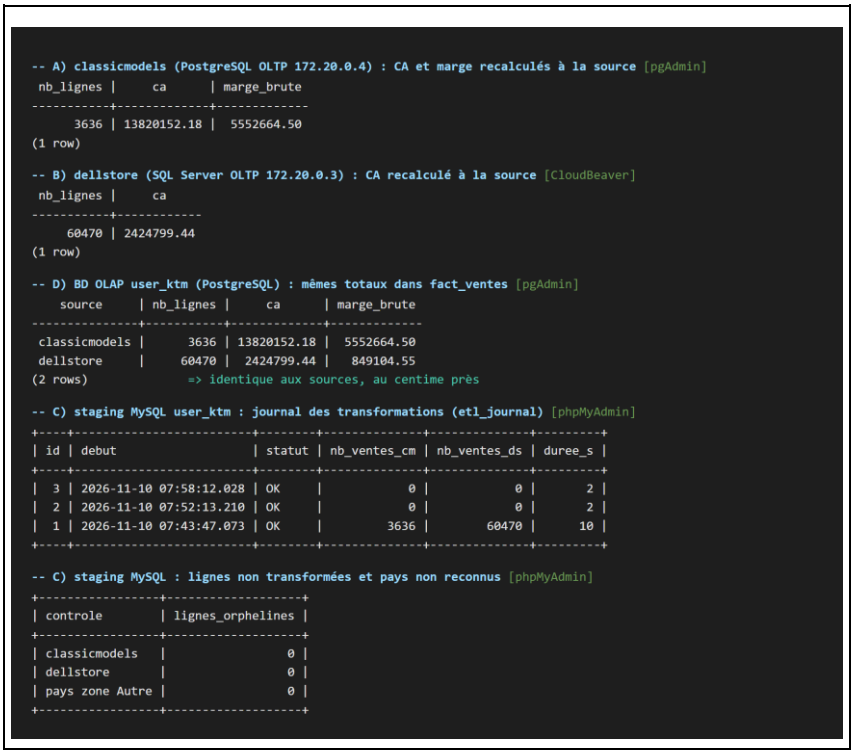


FIGURE 13 – Contrôles du script 04 sur les bases de l'école : totaux sources = OLAP, journal, lignes orphelines

Limites.

- La marge brute de dellstore repose sur un coût estimé (65 % du prix), faute de prix d'achat dans la source.
- Le prix dellstore est le prix catalogue au moment de l'intégration, car `orderlines` ne garde pas le prix payé.
- Une ligne supprimée physiquement dans une source n'est pas propagée : le CDC par date détecte les ajouts et les modifications, pas les suppressions.
- Les dimensions sont en SCD type 1 : un produit renommé l'est aussi dans l'historique.

A Fichiers fournis

Fichier	Contenu
`sql/01_olap_postgres.sql`	Création de l'étoile et génération de `dim_date`
`sql/02_staging_mysql.sql`	Tables de staging, de référence et de pilotage, vue `v_clients`
`sql/03_transform_mysql.sql`	Procédure `sp_transform`
`sql/04_controles.sql`	Requêtes de vérification post-chargement
`nifi/flux_projet_bi.json`	Flux NiFi complet (définition importable)
`powerbi/ProjetBI.pbip`	Projet Power BI (modèle, mesures, page de rapport)
`powerbi/mesures_dax.txt`	Mesures DAX